

Linux File Permissions Review and Remediation

Simulated Case Study

Project Overview

As part of a simulated security lab, I reviewed file and directory permissions for a research team working in `/home/researcher2/projects`. The goal was to identify permissions that did not match the organization's access requirements and correct them using Linux commands.

The review focused on removing unauthorized write access, securing an archived file, and restricting access to a directory intended for a single user. I used Linux commands to inspect the existing permissions and make the necessary changes while preserving authorized access.

Permission Review

I examined the permissions in the research team's project directory using `ls -la`, which displays detailed file information and includes hidden files. The listing showed the permissions assigned to the owner, group, and other users. Three entries required changes to meet the organization's access requirements.

```
cd /home/researcher2/projects
```

```
ls -la
```

Permission Review Findings

File or Directory	Original Permissions	Required permissions
<code>project_k.txt</code>	<code>-rw-rw-rw-</code>	<code>-rw-rw-r--</code>
<code>.project_x.txt</code>	<code>-rw--w----</code>	<code>-r--r-----</code>
<code>drafts/</code>	<code>drwx--x---</code>	<code>drwx-----</code>

The remaining files—`project_m.txt`, `project_r.txt`, and `project_t.txt`—already met the stated access requirements and did not need changes.

For example, the archived file `.project_x.txt` initially had the permission string `-rw--w----`. This identifies a regular file where the owner has read/write access, the group has write-only access, and others have no access. The letters `r`, `w`, and `x` represent read, write, and execute permissions, while a dash indicates that permission has not been granted.

Permission Changes

Remove unauthorized file-write access

The file `project_k.txt` initially allowed other users to read and modify its contents. Because the organization does not permit write access for others, I removed that permission while keeping the owner and group's existing access unchanged.

Command: `chmod o-w project_k.txt`

Resulting permissions: `-rw-rw-r--`

Restrict access to the archived file

The archived file `.project_x.txt` needed to remain readable by the owner and group without allowing either to modify it. I updated the permissions to make the file read-only for both categories while keeping it inaccessible to others.

Command: `chmod u=r,g=r,o= .project_x.txt`

Resulting permissions: `-r--r-----`

Secure the drafts directory

The drafts directory was intended for `researcher2` only. Its original permissions still allowed the group to traverse the directory, so I removed that access while preserving the owner's existing permissions.

Command: `chmod g-x drafts`

Resulting permissions: `drwx-----`

Verification and Outcome

After applying the permission changes, I reviewed the resulting permission strings to confirm that the files and directory matched the research team's access requirements.

The updated permissions removed write access for others from `project_k.txt`, made `.project_x.txt` readable but not writable by the owner and group, and restricted the drafts directory to `researcher2`.

These changes corrected the identified access-control issues while preserving the permissions needed for authorized research work.